SE3040 - Application Frameworks
Deployment Report - GreenRent

1. Introduction

This report describes the deployment process of the GreenRent full-stack application. GreenRent is a MERN-based system developed to help users discover eco-friendly rental properties, manage bookings, and receive AI-based recommendations. The system includes a React frontend, a Node.js (Express) backend, and a MongoDB database.

2. Deployment Architecture

The application is deployed using cloud-based services.

- Frontend: Hosted on Render
- Backend: Hosted on Render
- Database: MongoDB Atlas (cloud database)

The frontend communicates with the backend via REST API calls, and the backend interacts with the MongoDB database.

3. Backend Deployment

3.1 Platform Used

Render (Web Service)

3.2 Deployment Steps

1. The backend source code was pushed to the GitHub repository.
2. Logged into the Render dashboard.
3. Created a new Web Service.
4. Connected the GitHub repository.
5. Set the root directory to the server folder.
6. Added required environment variables.

7. Configured the start command:
   node src/server.js
8. Deployed the backend service.

3.3 Environment Variables

- MONGODB_URI
- JWT_SECRET
- NODE_ENV
- CLIENT_URL
- SMTP_USER
- SMTP_PASS
- GEMINI_API_KEY
- STRIPE_PUBLISHABLE_KEY
- STRIPE_SECRET_KEY

3.4 Live API URL

https://greenrent-gakv.onrender.com/

4. Frontend Deployment

4.1 Platform Used

Render (Web Service)

4.2 Deployment Steps

1. The frontend source code was pushed to the GitHub repository.
2. Connected the repository to Render.
3. Configured build settings:
   - Build Command: npm install && npm run build
   - Output Directory: dist
4. Deployed the frontend application.

4.3 Environment Variables

- VITE_API_URL

4.4 Live Application URL

https://greenrent-frontend.onrender.com/

5. Integration Between Frontend and Backend

The frontend communicates with the backend using RESTful API calls. Axios is used to send HTTP requests from the React application to the backend API.

The backend URL is stored as an environment variable in the frontend to ensure flexibility across environments. Cross-Origin Resource Sharing (CORS) is configured in the backend to allow requests from the deployed frontend.

6. Challenges Faced and Solutions

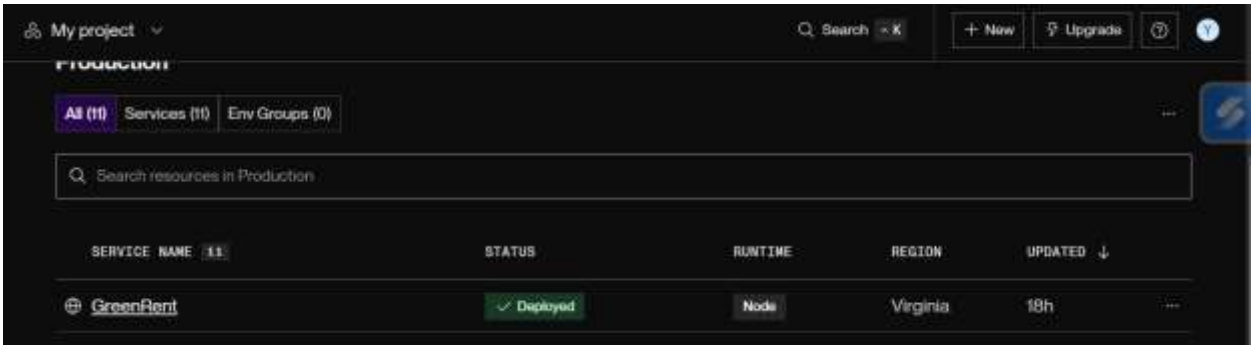
Issue 1: CORS errors during deployment
Solution: Configured CORS middleware in the backend to allow requests from the frontend domain.

Issue 2: Environment variables not loading correctly
Solution: Verified and reconfigured environment variables in the Render dashboard.

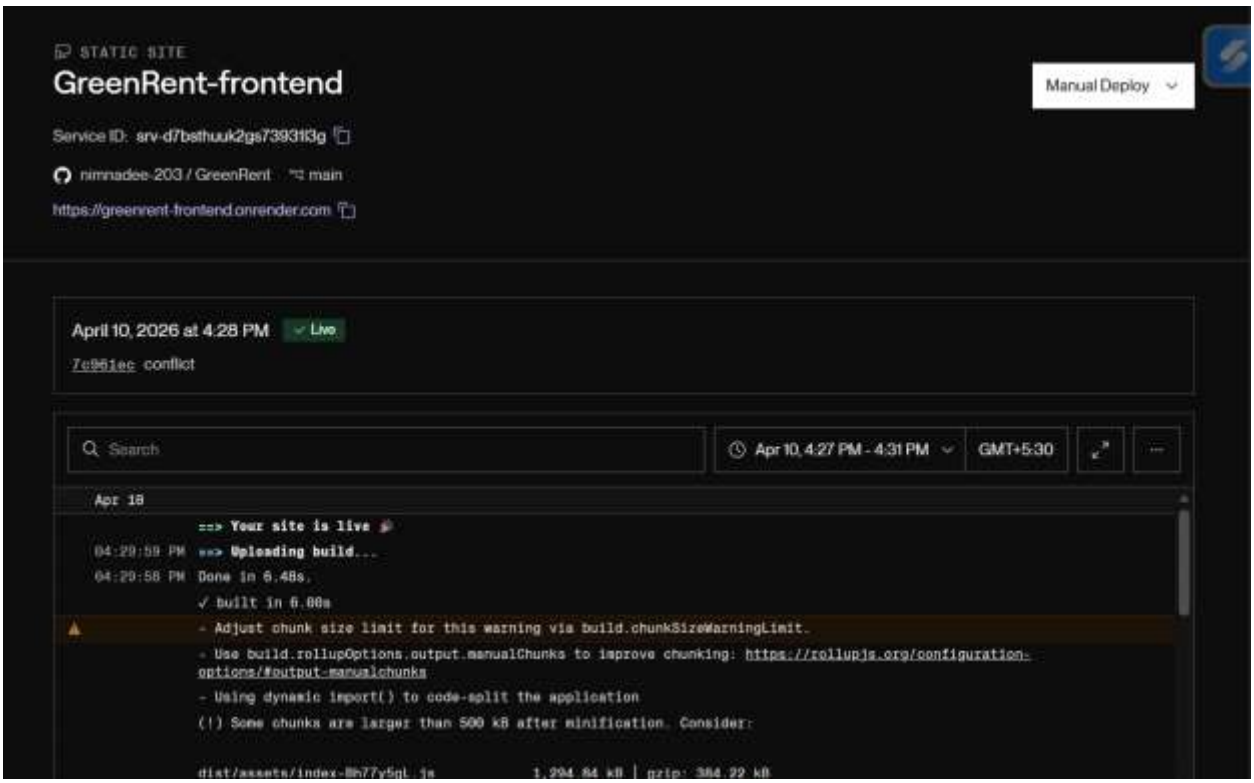
Issue 3: Frontend not connecting to backend API
Solution: Updated the API base URL in the frontend to use the deployed backend URL instead of localhost.

7. Deployment Evidence

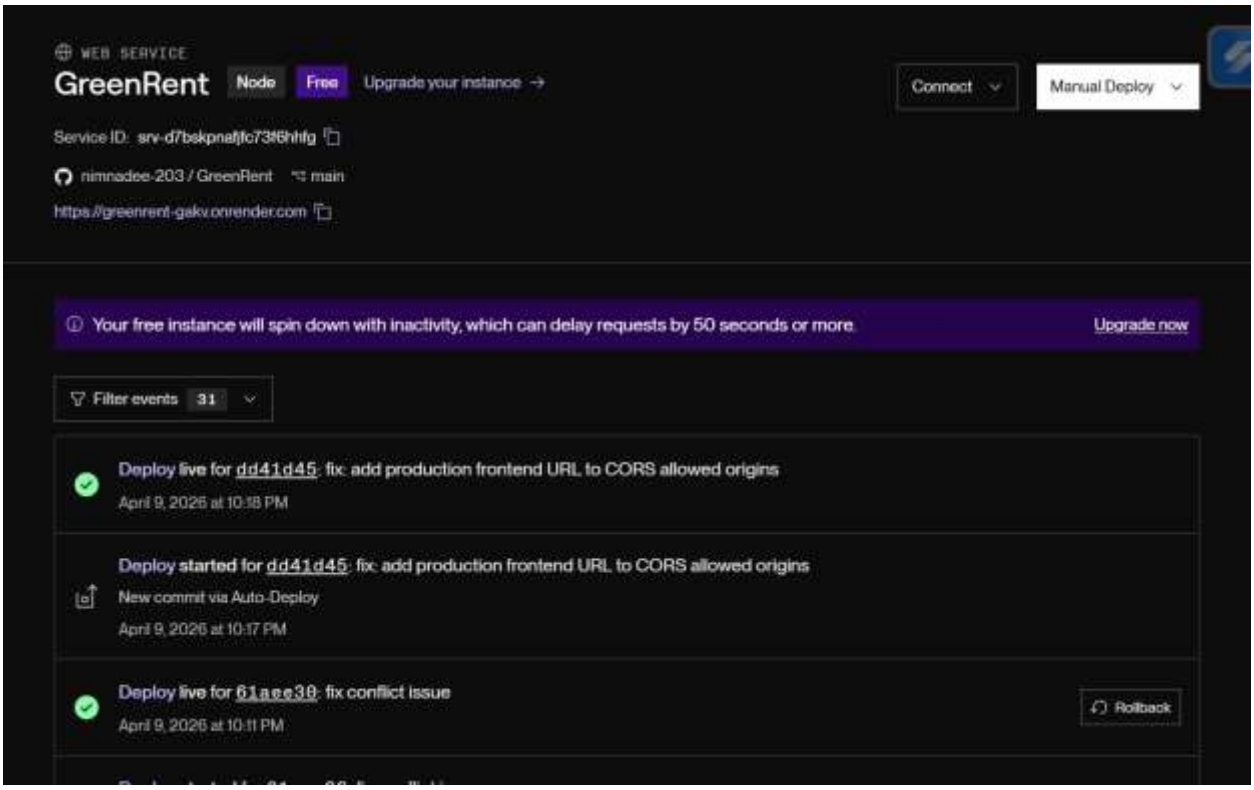
- Render Dashboard deployment history



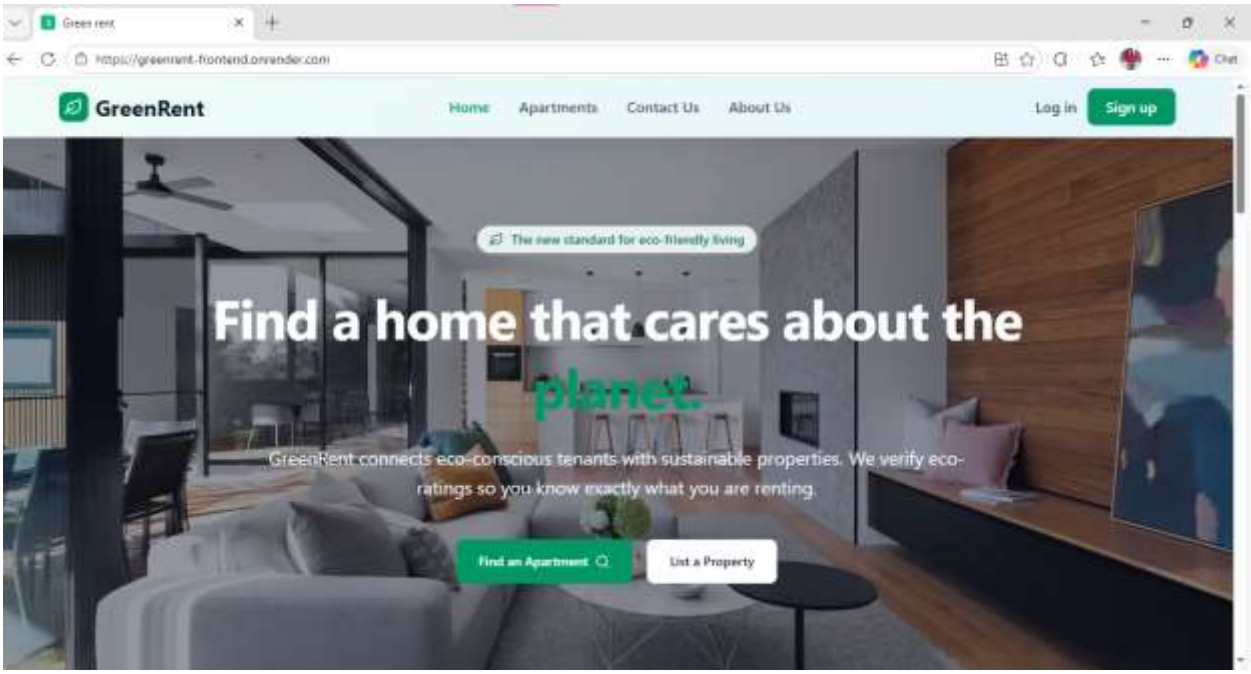
- Render frontend deployment dashboard



- Render backend service dashboard



- Running frontend application in browser



8. Conclusion

The GreenRent application was successfully deployed using Render for both frontend and backend services. The system is fully functional and accessible online, supporting key features such as authentication, property listing, booking, eco-rating, and recommendations.